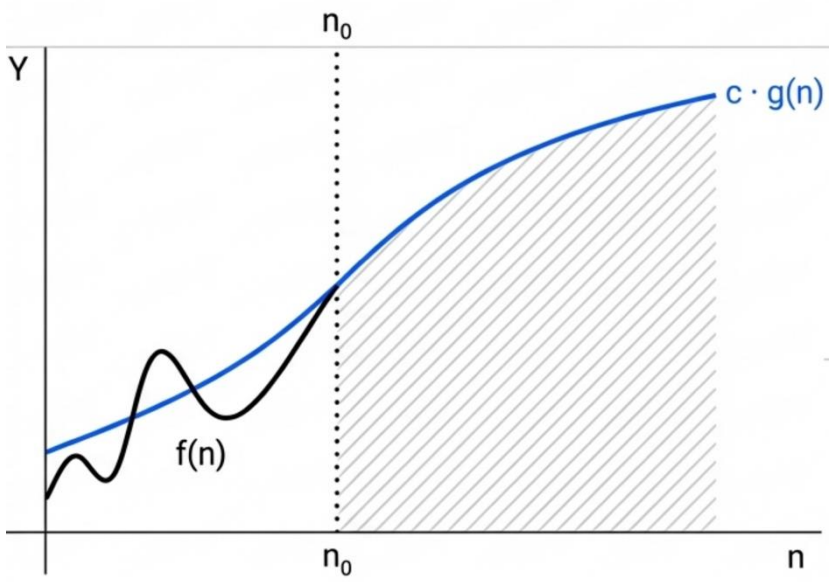
Universidad del Rosario | Escuela de Ciencias e Ingeniería | MACC

Complejidad algorítmica



Objetivos

- Comprender y aplicar rigurosamente las notaciones asintóticas (O, Ω, Θ) para acotar el crecimiento del tiempo de ejecución de un algoritmo en función del tamaño de su entrada (n).

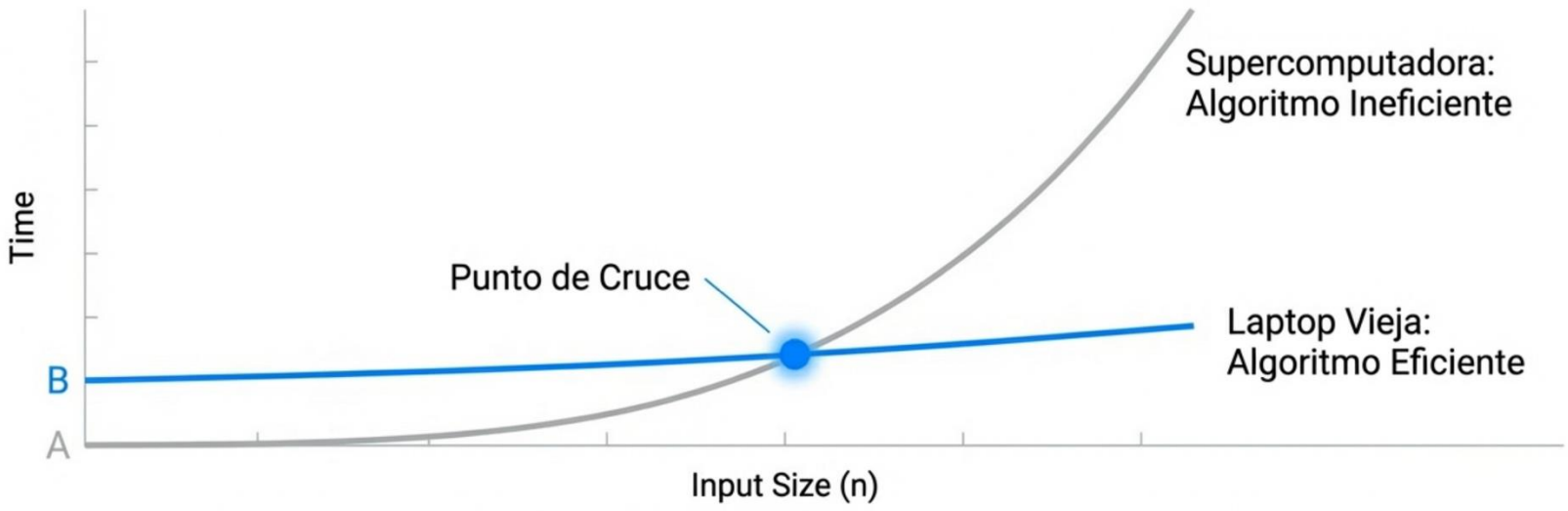
- Desarrollar la capacidad de mapear operaciones de código fuente a ecuaciones polinomiales de costo de tiempo utilizando el modelo computacional RAM, identificando términos dominantes e ignorando constantes de bajo orden.

- Distinguir claramente entre el análisis del peor caso (cota superior), el mejor caso (cota inferior) y el caso promedio, utilizando el algoritmo de ordenamiento por inserción (*Insertion Sort*) como objeto de estudio principal.

- Evaluar cómo las decisiones de implementación y el uso de las estructuras de datos en C++ moderno (como arreglos contiguos versus estructuras enlazadas) afectan la complejidad teórica y el rendimiento real debido a los patrones de acceso a memoria.

Motivación

El hardware moderno, con sus múltiples núcleos y jerarquías de caché L1/L2/L3, es extraordinariamente rápido. Esta velocidad a menudo enmascara la ineficiencia algorítmica durante las fases de desarrollo y pruebas con conjuntos de datos pequeños. Sin embargo, existe una mentira común en la ingeniería de software: creer que los problemas de rendimiento se resuelven escalando el hardware ("*throw hardware at it*").

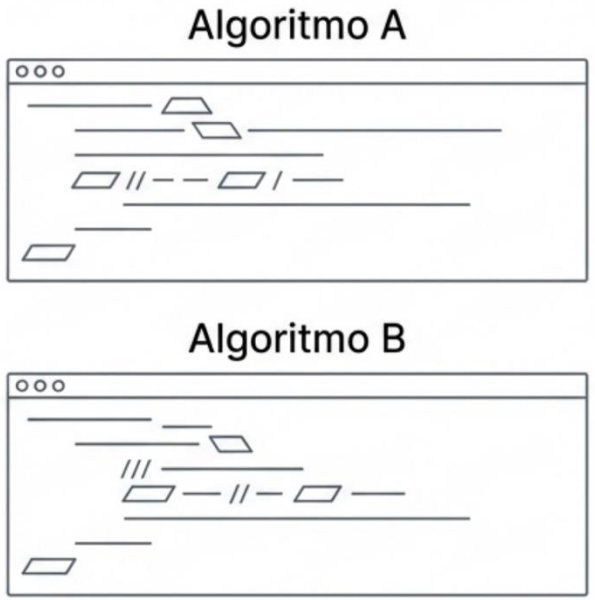


Supercomputadora: 10,000 millones de instr/seg.
Algoritmo lento: $2n^2$

Laptop Vieja: 10 millones de instr/seg.
Algoritmo rápido: `50n log n`

El problema de la medición

Tenemos dos algoritmos que resuelven exactamente el mismo problema. ¿Cómo decidimos cuál es el mejor?



La intuición inicial dicta usar un cronómetro o contar cada paso del código. En Ciencias de la Computación, ambas aproximaciones son engañosas.

El problema de la medición

Ejecutar el algoritmo y medir cuántos milisegundos tarda en finalizar.

El tiempo de ejecución real no mide el algoritmo. Mide el entorno.

El problema de la medición

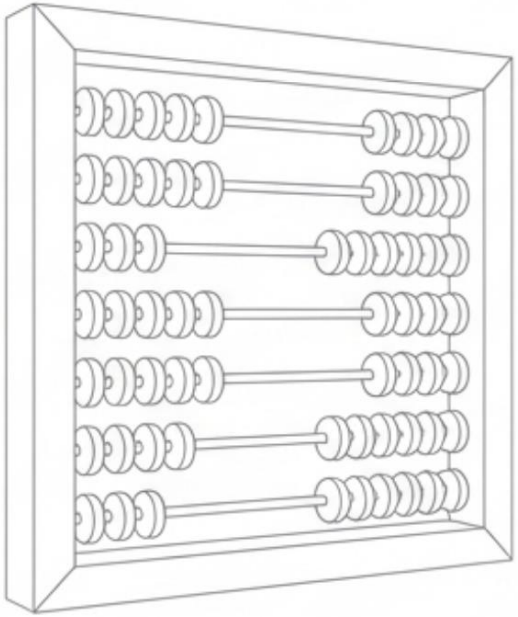
El tiempo cronológico (T) está contaminado por variables externas. Ejecutar el mismo código dos veces en la misma máquina arrojará tiempos distintos debido a la carga dinámica del procesador.

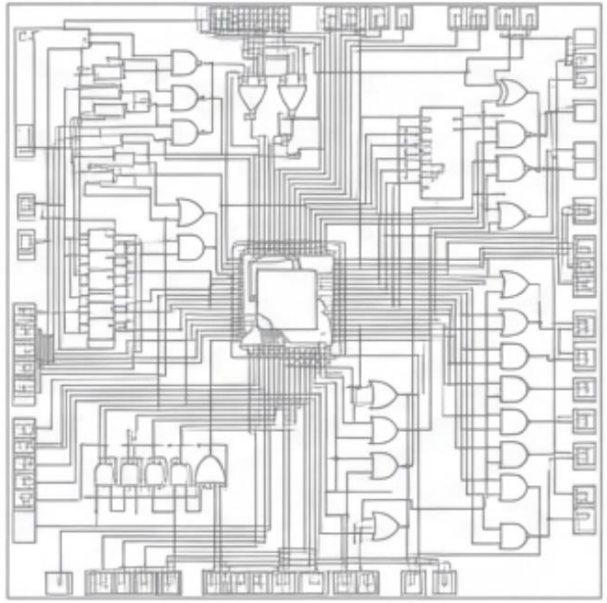
No podemos juzgar la lógica matemática basándonos en fluctuaciones eléctricas.

El problema de la medición

Contar exactamente cuántas operaciones primitivas (sumas, asignaciones) realiza el código usando el Modelo RAM (Random-Access Machine).

Es teóricamente independiente del hardware, pero matemáticamente insostenible en la práctica.





El problema de la medición

	Independiente del hardware	Matemáticamente simple	Predice a gran escala
Cronómetro	✗	✓	✗
Conteo Exacto	✓	✗	✓
Lo que necesitamos	?	?	?

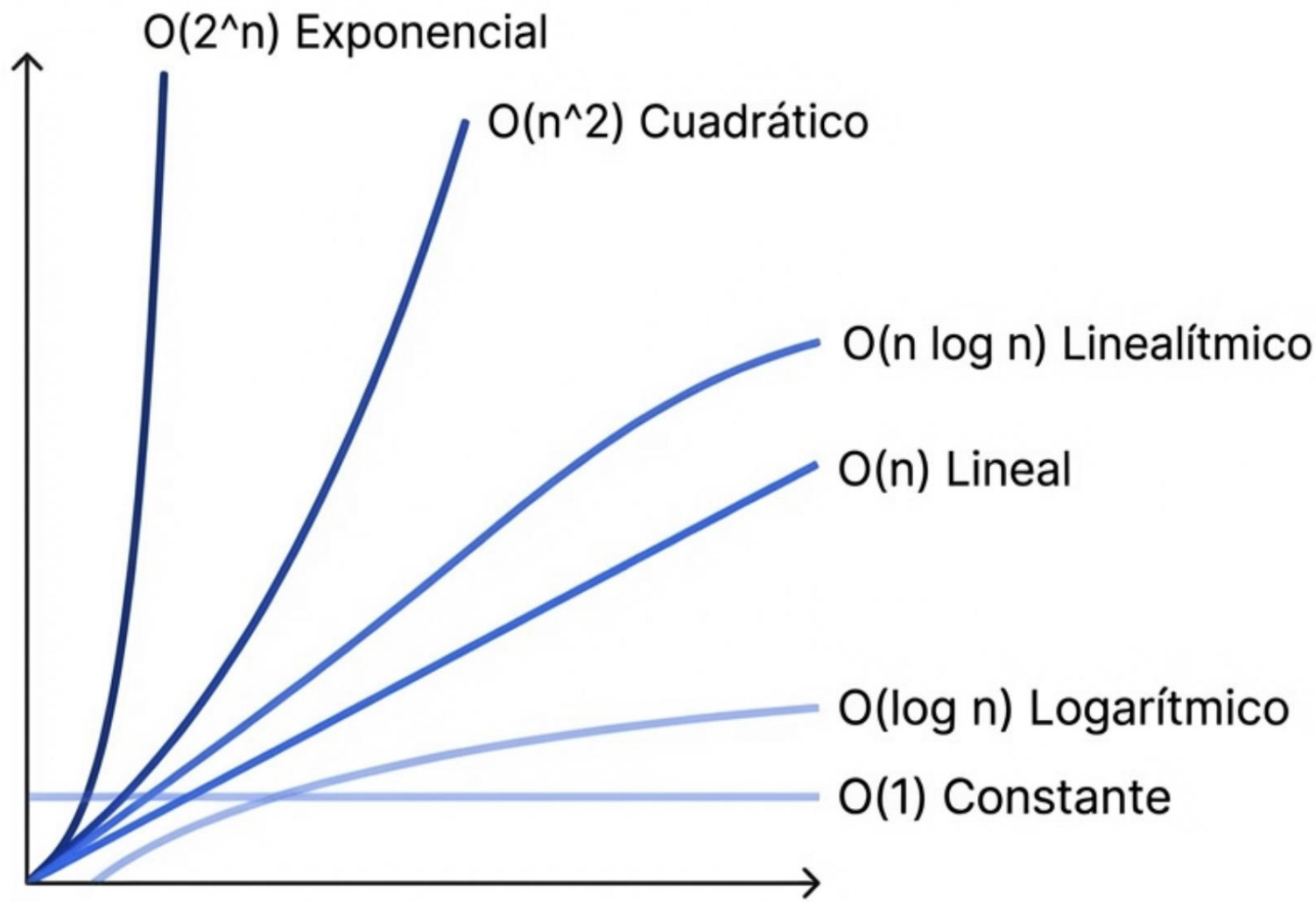
El problema de la medicón

En Ciencias de la Computación, no importa cómo se comporta un algoritmo con 10 o 100 datos.
Nos importa cómo reacciona cuando el volumen de datos crece hacia el infinito.

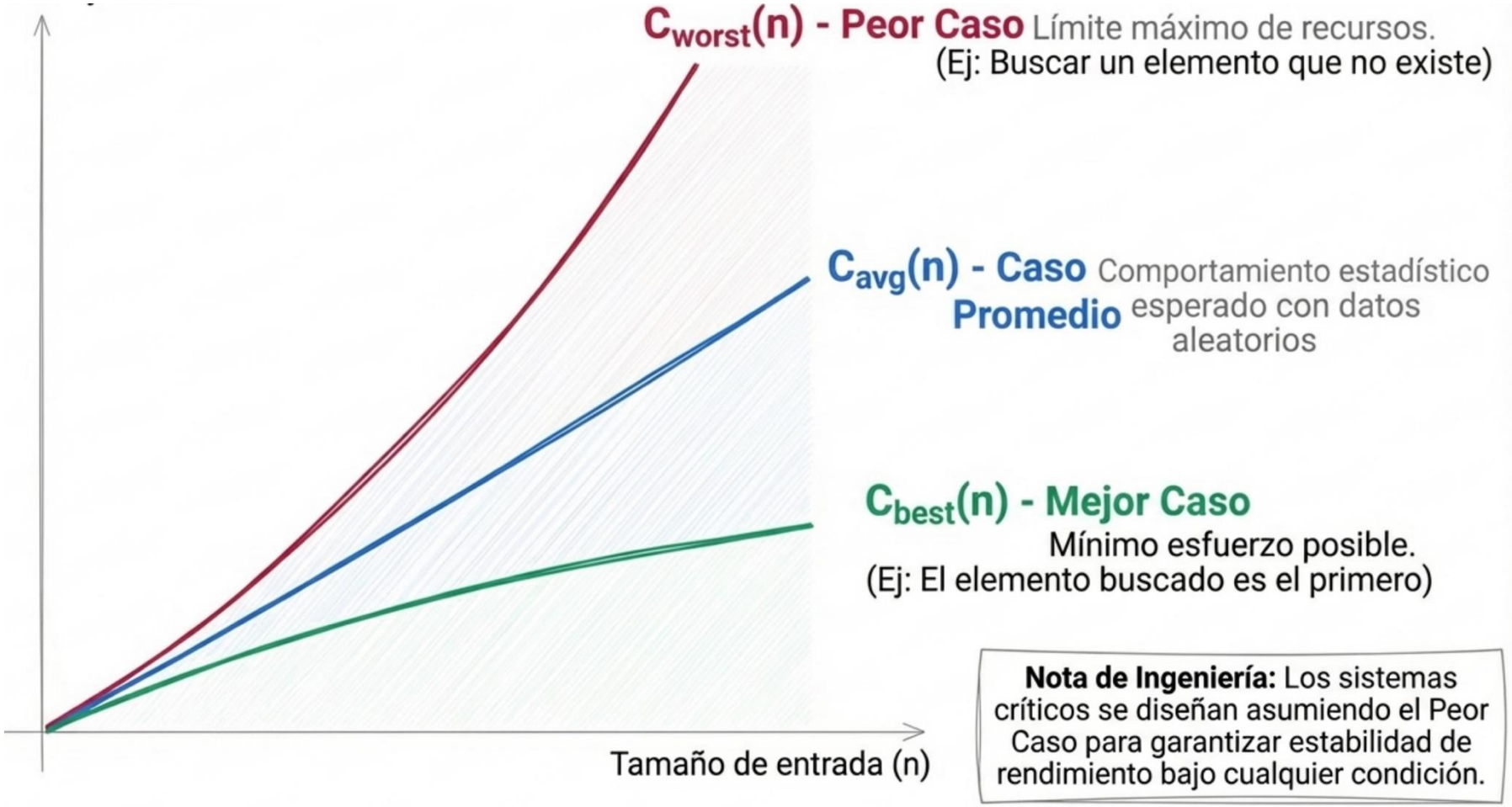
$$n \to \infty$$

A esto se le conoce como la Tasa de Crecimiento Asintótico.

Al abstraer el tiempo, podemos clasificar universalmente cualquier algoritmo en Clases de Eficiencia. Esta es la verdadera medida del éxito.



Tiempo de ejecución



Modelo RAM

Ejecución Secuencial:
Una instrucción a la vez, sin concurrencia.

Costo Constante:
Cada operación fundamental toma un tiempo fijo (c_i).

Agnóstico al Hardware:
Medimos pasos lógicos, no milisegundos.

```
for i = 1 to n
    key = A[i]
    A[i+1] = key
```

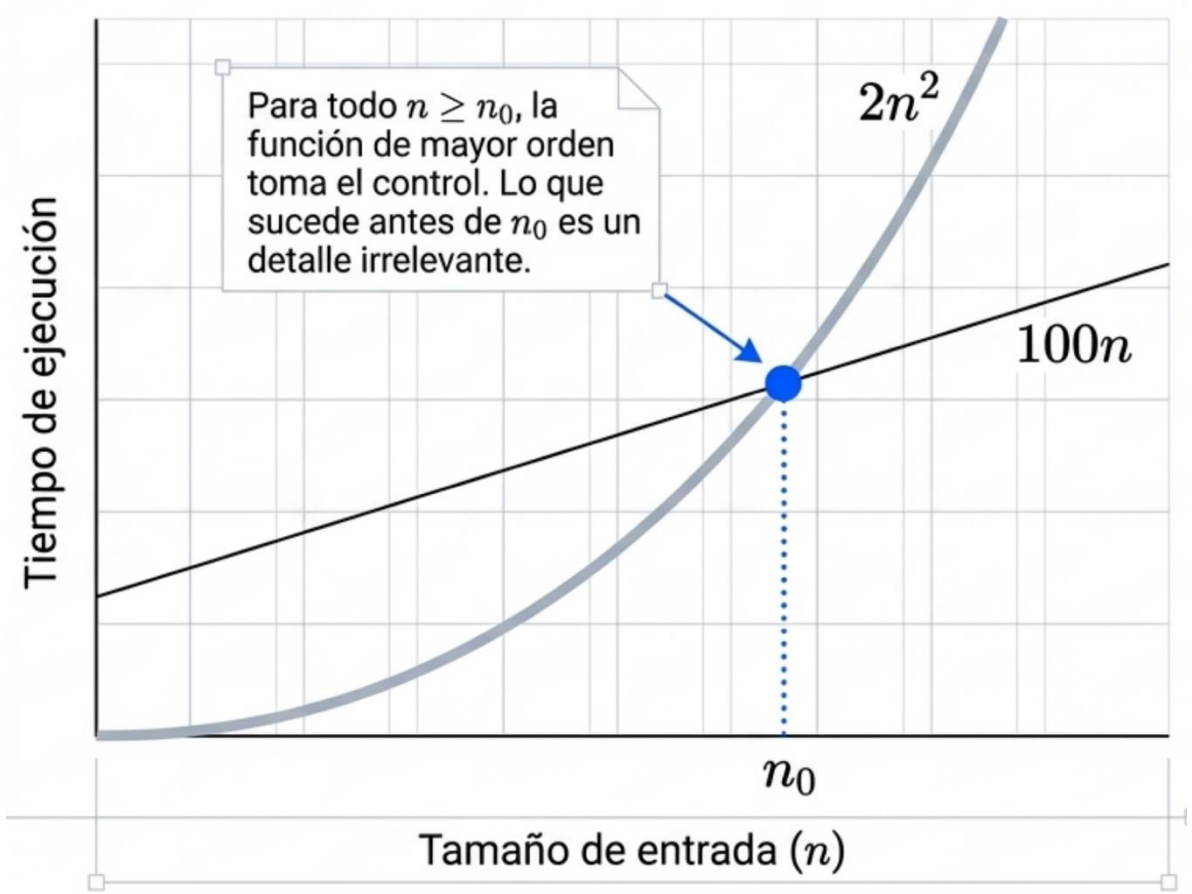
- `for i = 1 to n` → Costo: c_1 (n veces)
- `key = A[i]` → Costo: c_2 (n-1 veces)
- `A[i+1] = key` → Costo: c_3 (n-1 veces)

$$T(n) = c_1 n + c_2(n-1) + c_3(n-1)$$

El umbral asintótico

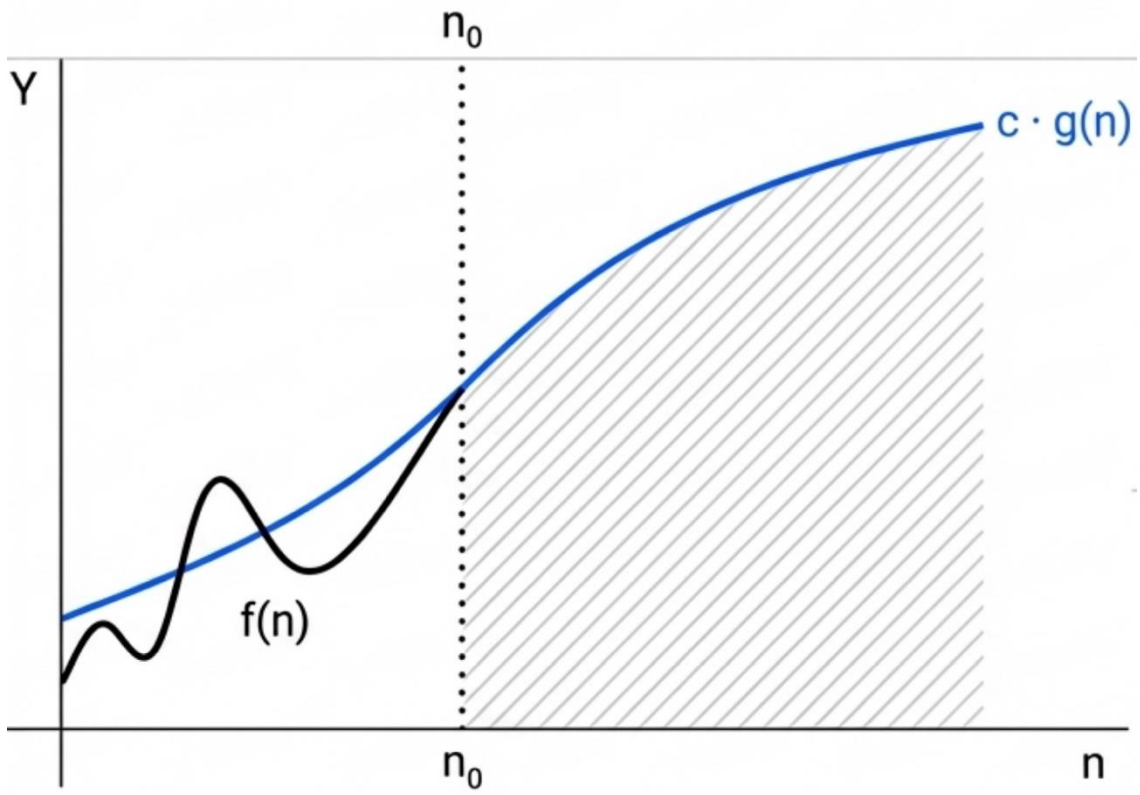
En el análisis asintótico, ignoramos los términos de menor orden y los coeficientes constantes.

Porque para cualquier función con un orden de crecimiento mayor, siempre existe un tamaño de entrada a partir del cual dominará absolutamente el tiempo de ejecución.



Notación big-O

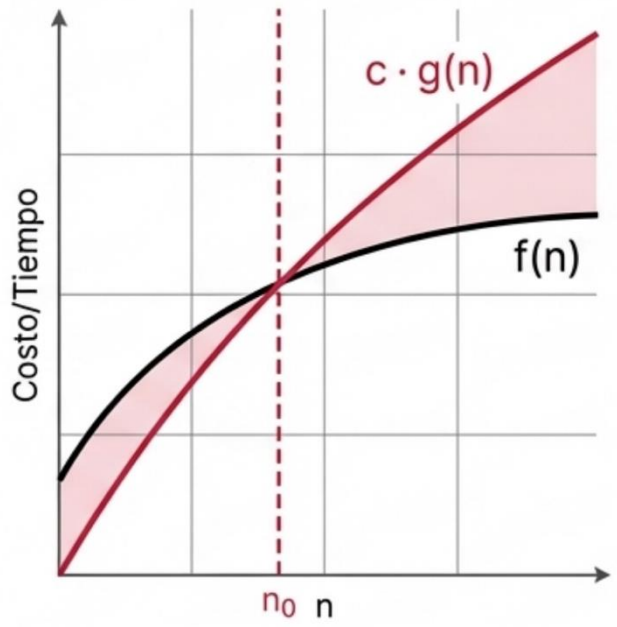
$$O(g(n)) = \{ f(n) : \exists\, c, n_0 > 0 \text{ tal que } 0 \leq f(n) \leq c \cdot g(n) \text{ para todo } n \geq n_0 \}$$

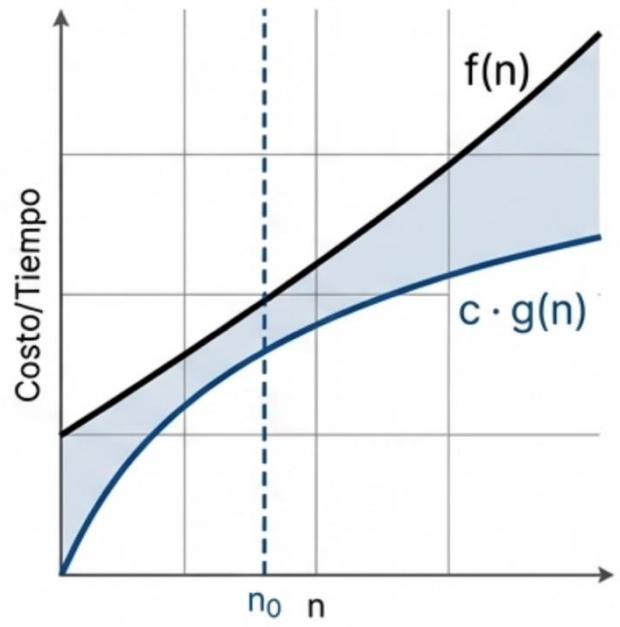


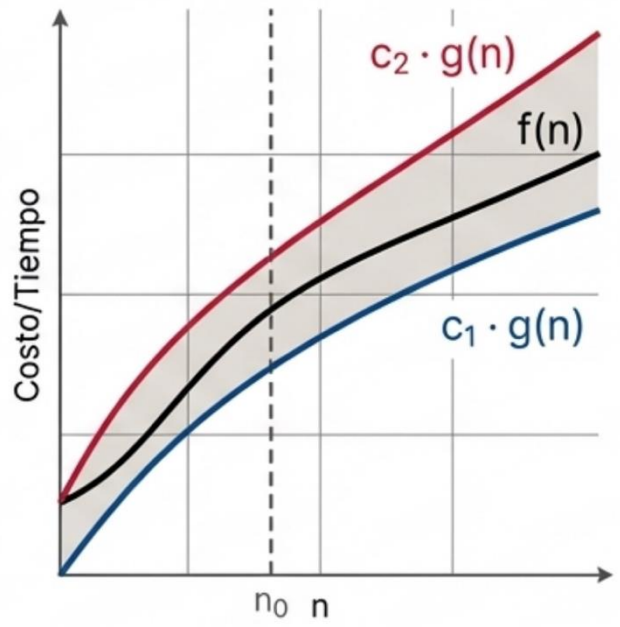
Big-O no es una función única, es un conjunto de funciones. El área sombreada representa todas las funciones acotadas por arriba por c.g(n).

Límites Estrictos

Big-O (O) - Cota Superior	Big-Omega (Ω) - Cota Inferior	Big-Theta (Θ) - Cota Ajustada
$f(n) \leq c \cdot g(n)$ para $n \geq n_0$	$f(n) \geq c \cdot g(n)$ para $n \geq n_0$	$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$







La notación asintótica

Notación Asintótica	Equivalente (Números Reales)	Significado Intuitivo
$O(g(n))$	$a \leq b$	Crece igual o más lento.
$\Omega(g(n))$	$a \geq b$	Crece igual o más rápido.
$\Theta(g(n))$	$a = b$	Crecen a la misma tasa exacta.
$o(g(n))$ [Little-o]	$a < b$	Crece estrictamente más lento (peso relativo cero).
$\omega(g(n))$ [Little-omega]	$a > b$	Crece estrictamente más rápido.

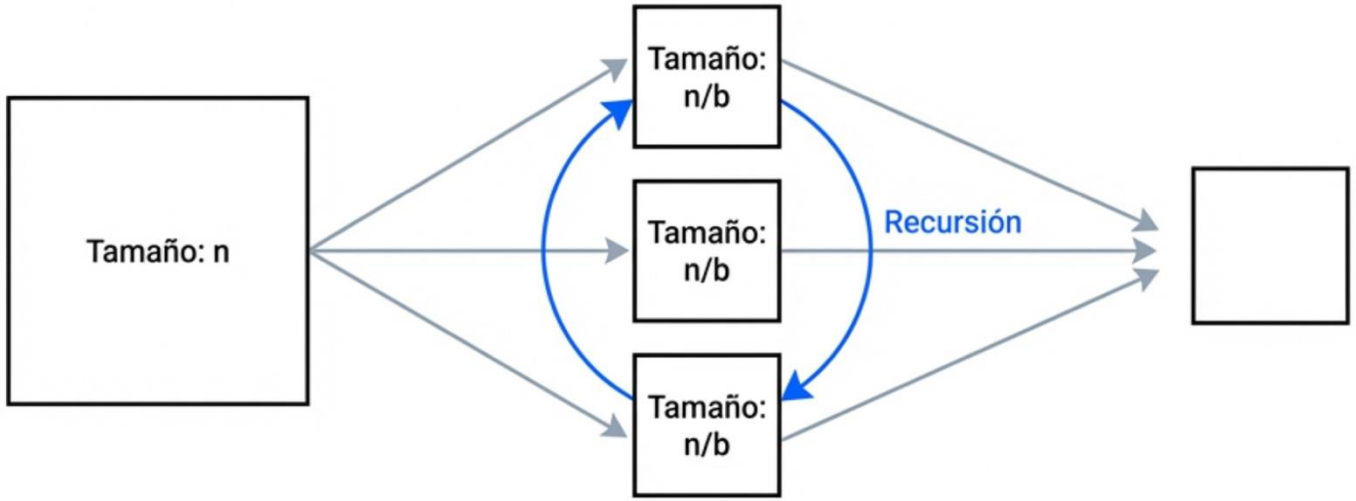
Nota de Precaución: A diferencia de los números reales (donde $a < b$, $a = b$, o $a > b$ siempre es cierto), no todas las funciones matemáticas son comparablemente asintóticas (ej. n vs $n^{1+\sin n}$).

Analicemos INSERTIONSORT

```
INSERTION-SORT(A, n)
for i = 2 to n
    key = A[i]
    // Insertar A[i] en el subarreglo ordenado A[1 ... i-1]
    j = i - 1
    while j > 0 and A[j] > key
        A[j + 1] = A[j]
        j = j - 1
    A[j + 1] = key
```

Analizando la recursión

El tiempo de ejecución de algoritmos recursivos (como Merge Sort) se modela mediante Ecuaciones de Recurrencia.



$$T(n) = aT(n/b) + f(n)$$

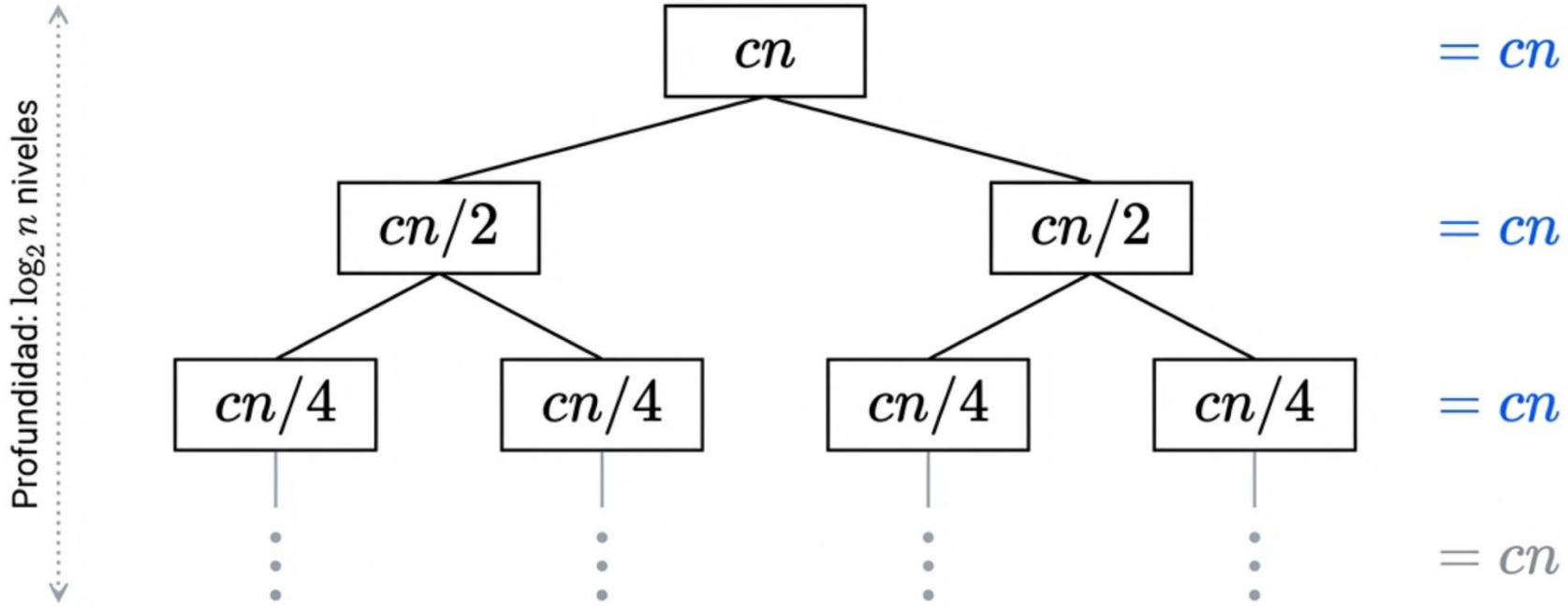
a: Cantidad de subproblemas creados.

n/b: Tamaño de cada subproblema individual.

f(n): Costo de la división inicial más el costo de combinar los resultados (Driving Function).

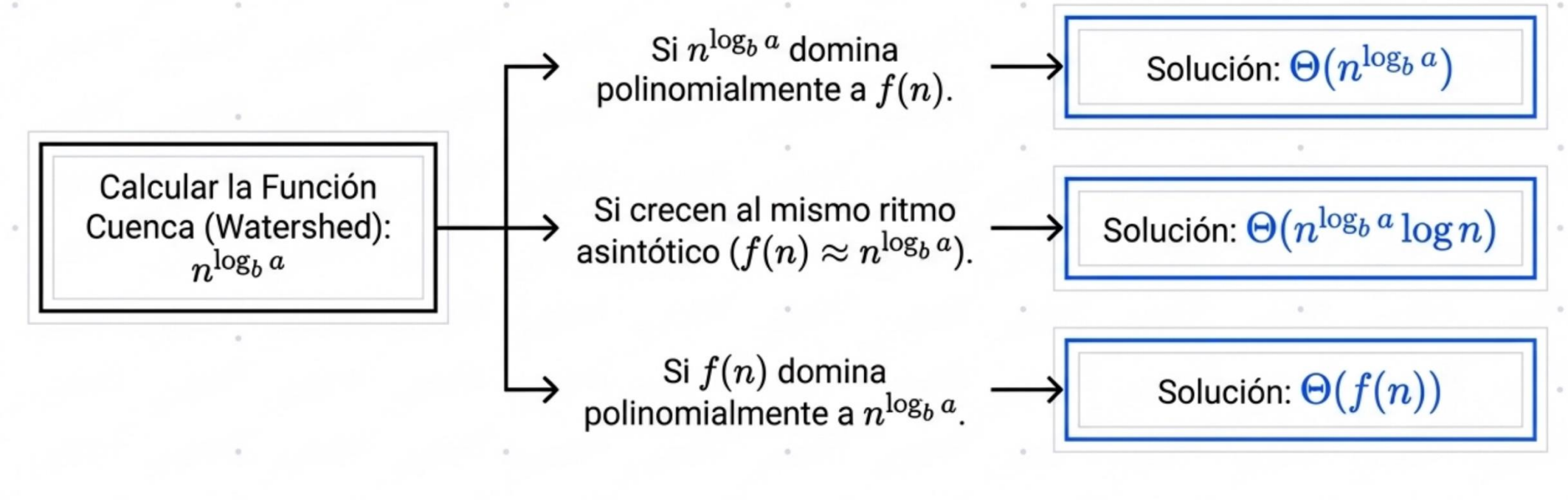
Analizando la recursión

Resolviendo $T(n) = 2T(n/2) + cn$ expandiendo la recurrencia.



Suma Total: Costo por nivel (cn) $\times$ Número de niveles $(\log_2 n) = \Theta(n \log n)$

Teorema maestro



El efecto de la estructura de datos

```
// Función genérica para medir el tiempo de recorrido de cualquier contenedor
template <typename Contenedor>
long long medirRecorrido(const Contenedor& c) {
    auto inicio = std::chrono::high_resolution_clock::now();

    volatile long long suma = 0;
    // Recorrido O(n) para cualquier contenedor que soporte iteradores
    for (const auto& elemento : c) {
        suma += elemento;
    }

    auto fin = std::chrono::high_resolution_clock::now();
    return std::chrono::duration_cast<std::chrono::microseconds>(fin - inicio).count();
}
```

Resumen

El análisis de complejidad abstrae la velocidad del procesador y la arquitectura. Modelamos el tiempo de ejecución $T(n)$ en función del tamaño de la entrada n, asumiendo un costo constante c_k para las instrucciones primitivas.

Utilizamos $\boldsymbol{O}$ **(Big-O)** y $\boldsymbol{\Omega}$ **(Omega)** para establecer las fronteras globales de un *algoritmo en general*, independientemente de su entrada. (Ej. *Insertion Sort* es $O(n^2)$ y $\Omega(n)$).

A medida que $n \to \infty$, los términos de orden inferior y los coeficientes constantes pierden relevancia. Un algoritmo con cota superior $O(n \log n)$ siempre superará a uno $O(n^2)$ para entradas suficientemente grandes, sin importar el hardware.

La contigüidad espacial (caché L1) y la ausencia de sobrecarga por llamadas al sistema (Call Stack) hacen que algoritmos asintóticamente ineficientes dominen empíricamente cuando n es minúsculo. Esta es la base de la optimización híbrida en la STL de C++.

En la teoría pura, recorrer un arreglo de n elementos y recorrer una lista enlazada de n nodos tienen exactamente la misma complejidad temporal asintótica: $\boldsymbol{O(n)}$. En el modelo RAM, ambos algoritmos deberían tardar lo mismo. En la ingeniería de software real, esta suposición colapsa debido a la jerarquía de memoria del hardware moderno (Registros → Caché L1/L2/L3 → RAM).